

LLM 预训练之 Embedding

Token Embedding / Unembedding / LM Head • 深度研究手册

从离散 Token ID 到连续残差流，再从 Hidden State 回到 Vocabulary：参数经济、Weight Tying、Rare Token、Softmax Bottleneck、Vocabulary Expansion 与最新 Per-Layer Embedding

核心视角：Embedding 是模型的“Token Interface”，不是无关紧要的查表层

Research cut-off: 2026-08-13 | 01-B-Agent

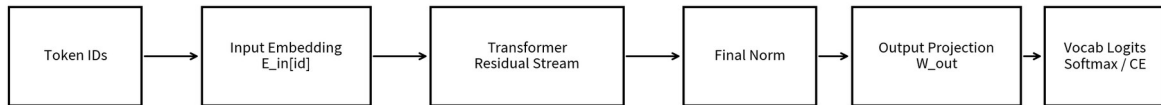
执行摘要：Embedding 决定模型如何“读 token”与“说 token”

对 decoder-only LLM，输入端用 embedding table 把 token id 映射到 d_{model} 维向量；输出端用 LM Head / unembedding 把最终 hidden state 投影回 V 维 vocabulary logits。两端看起来只是两个矩阵乘法，但它们同时决定参数量、训练梯度的统计结构、低频 token 的学习速度、词表扩展成本、分布式切分方式，以及 next-token distribution 的表达上限。

核心判断	Know-why	工程含义
Embedding 是双向接口，不只是 lookup	输入端编码 token identity，输出端实现 V -way 分类	必须联合设计 E_{in} 、LM Head、tie policy 与 tokenizer
Vocab size 会直接放大参数块	单矩阵参数量为 $V \times d$	128K-262K vocab 时常是数亿至十多亿参数
Weight tying 改变优化动力学	input lookup gradient 与 output CE gradient 统计完全不同	省参数不等于免费；应以模型规模和实验决定 tied/untied
Rare token 是结构性长尾问题	输入 embedding row 的直接更新次数跟 token 频率相关	监控 per-token exposure / norm / loss，而非只看总体 loss
LM Head 可能成为表达与梯度瓶颈	hidden dimension d 通常远小于 vocab V	小模型与超大词表组合尤其值得评估 softmax bottleneck
Vocabulary extension 是 continued-pretraining 问题	新 token 没有历史 embedding 几何	初始化方法需用短 CPT / downstream convergence 评估

最重要的 Know-why Tokenizer 决定“有哪些符号”；Embedding 决定“这些符号以什么几何进入模型”；LM Head 决定“模型如何把内部状态变成下一个符号的概率”。三者必须共同设计。

Embedding 是离散 token 空间与 Transformer 连续表示空间之间的双向接口



Embedding 不只是查表：它定义 token 身份如何进入连续空间；LM Head 定义连续状态如何回到离散词表。两端是否共享参数会改变优化动力学。

图 1 Embedding/LM Head 位于离散 token 空间与连续 Transformer 表示空间的两端。

1. 基础数学：Embedding 到底在计算什么

设 vocabulary 大小为 V ，hidden size 为 d ，输入 embedding matrix 为 $E_{\text{in}} \in \mathbb{R}^{V \times d}$ 。对 token id i ，输入向量只是第 i 行 $e_i = E_{\text{in}}[i]$ 。对 sequence $x = (x_1, \dots, x_L)$ ，lookup 后得到 $X \in \mathbb{R}^{L \times d}$ ，再进入 Transformer。

输出端设 $W_{\text{out}} \in \mathbb{R}^{V \times d}$ ，最终 hidden state $h_t \in \mathbb{R}^d$ ，则 logits $z = W_{\text{out}} h_t$ （省略 bias）；softmax 得到 $p(v|x_{<t})$ ，再与目标 token 做 cross entropy。

对象	Shape	主要角色	梯度特点
Input embedding E_{in}	$V \times d$	token id $\rightarrow$ vector	lookup 路径只直接更新 batch 中出现的 rows
Hidden state h	$L \times d$	上下文计算后的状态	通过所有 Transformer 层连续更新
Output / unembedding W_{out}	$V \times d$	hidden $\rightarrow$ vocab logits	full softmax 下几乎所有 vocabulary rows 每个 target 都收到 CE gradient

对象	Shape	主要角色	梯度特点
Tied matrix E	$V \times d$	同时承担 input + output	两类梯度在同一参数上叠加

对输出 row w_j ，单个 target y 的梯度为 $\partial L / \partial w_j = (p_j - 1[j=y])h$ 。也就是说：即便 token j 没有出现在当前输入中，只要采用完整 softmax，它作为输出类别仍通常收到梯度。这个事实是理解 weight tying 与 rare-token 训练动态的关键。

2. 参数经济： $V \times d$ 为什么越来越重要

Embedding 的参数复杂度是 $O(Vd)$ 。当 vocabulary 从 32K 扩展到 128K、151K、262K 时，即使 d 不变，输入/输出矩阵也线性膨胀。是否 tying 会直接决定是否需要一份还是两份 $V \times d$ 参数。

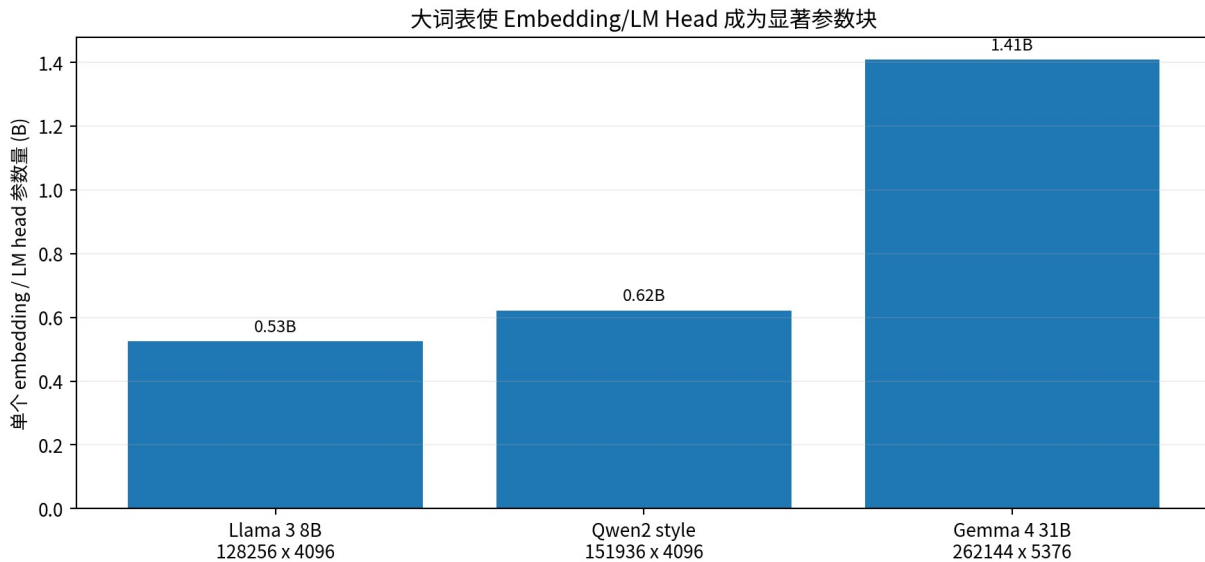


图 2 公开模型尺度下，单个 token embedding / output head 已经可以达到 0.5B-1.4B 参数。

公开架构例	Vocab / hidden	Tie policy	Embedding 参数含义
Llama 3 8B	128,256 / 4,096	Untied	约 0.525B input + 0.525B output; 官方实现分别定义 tok_embeddings 与 output [18][19]
Qwen2 config	151,936 / 4,096	Untied	单矩阵约 0.622B; Hugging Face Qwen2 config 明确 tie_word_embeddings=False [20]
Gemma 4 31B	262,144 / 5,376	Tied	单 shared embedder 约 1.409B; 技术报告表 1 给出约 1.410B embedder [14]
Gemma 4 E2B/E4B	262K vocab	主表 + Per-layer embedding	报告称额外 per-layer embedder 分别约 2.34B / 2.82B 参数 [14]

工程判断 词表扩大带来的“更少 token”收益，要与更大的 Embedding/LM Head、训练通信、checkpoint、optimizer state 与输出 projection 成本一起算账。Tokenizer 设计与 Embedding 参数预算不可分开。

3. Scale / Initialization / Norm：输入向量的数值尺度

原始 Transformer 在 embedding 层把 lookup 向量乘以 $\sqrt{d_{\text{model}}}$ ，同时共享 embedding 与 pre-softmax 权重。该做法是一个特定初始化/尺度约定，而不是所有现代 decoder LLM 的统一规则。[1]

设计点	常见选择	为什么重要	验证方式
Initialization std	例如 Normal(0, 0.02) 或 fan-based	决定训练最初 token vector 范数与 logits 尺度	首 100-1000 steps 的 activation / grad norm
Embedding scaling	1、 $\sqrt{d}$ 、learned scale	影响进入 residual stream 的幅值	embedding RMS vs block input RMS
Padding row	固定 0 / no grad	避免 PAD 产生语义和梯度污染	padding row norm 应稳定
Norm control	无、RMS/LayerNorm、norm calibration	过大/过小 token norm 会改变 logits 与优化	per-frequency bucket norm histogram
Weight decay	对 embedding decay / exclude	稀有 token 长时间不更新时 decay 会有额外影响	rare token norm drift

不要把“embedding norm 越大 = token 更重要”当作简单语义解释。norm 同时受 token frequency、output classification gradient、weight tying、optimizer、normalization 与 initialization 影响。尤其 tied 模型中，输入语义与输出决策边界共享同一几何。

4. Weight Tying：经典结论与现代再评估

Press & Wolf 与 Inan 等在 2016-2017 年分别从经验和理论角度支持将输入 embedding 与输出 projection 共享：可减少参数，并在当时的语言模型上改善 perplexity。原始 Transformer 也采用共享矩阵。[2][3][1]

Weight Tying 的本质不是“省一张表”，而是改变梯度流

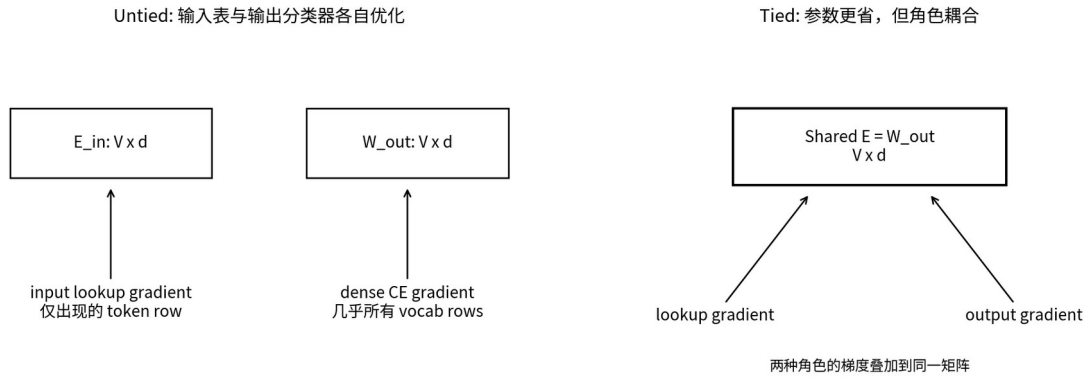


图 3 Tied 与 untied 的真正差异在梯度角色：lookup 梯度与全词表分类梯度是否作用于同一矩阵。

Tied 的收益	Tied 的代价 / 假设
参数量少一份 $V \times d$ ；对大词表尤其显著	假设“读 token”的空间与“把 hidden 分类为 token”的空间可以兼容
输出梯度给低频 token row 提供额外更新信号	output gradient 通常比 input lookup gradient 更密集，可能主导共享空间
经典 RNN/Transformer 工作常见更好 perplexity	2024 研究指出 tying 与 distributional hypothesis 存在隐含联系，并非所有符号系统都适合 [7]
部署时减少 checkpoint / memory	现代 Llama/Qwen 等常选择 untied，说明大模型实践并未收敛到唯一答案 [18][20]

2026 年 “Weight Tying Biases Token Embeddings Towards the Output Space” 报告：tied embedding 更接近 untied 模型的 output embedding，而不是 input embedding；作者将其归因于训练早期 output gradient 的支配，并通过输入梯度缩放提供因果证据。这是近期机制研究，仍应与具体大规模生产模型的 ablation 区分。[16]

Know-how 不要用“参数更少”单独决定 tie_word_embeddings。至少比较：同总 compute、同 token budget、同 tokenizer 下的 tied vs untied；记录 embedding/output gradient norm ratio、rare token loss、validation loss、下游能力和内存收益。

5. Rare Token Problem：Embedding 学习不是均匀的

Subword tokenizer 仍然产生高度长尾的 token frequency。输入 embedding row 只有在对应 token 出现在 input 时才得到直接 lookup gradient；因此低频 token 的累计输入更新次数可能比高频 token 少几个数量级。

Rare Token Problem: 输入 embedding 的直接更新机会高度不均衡

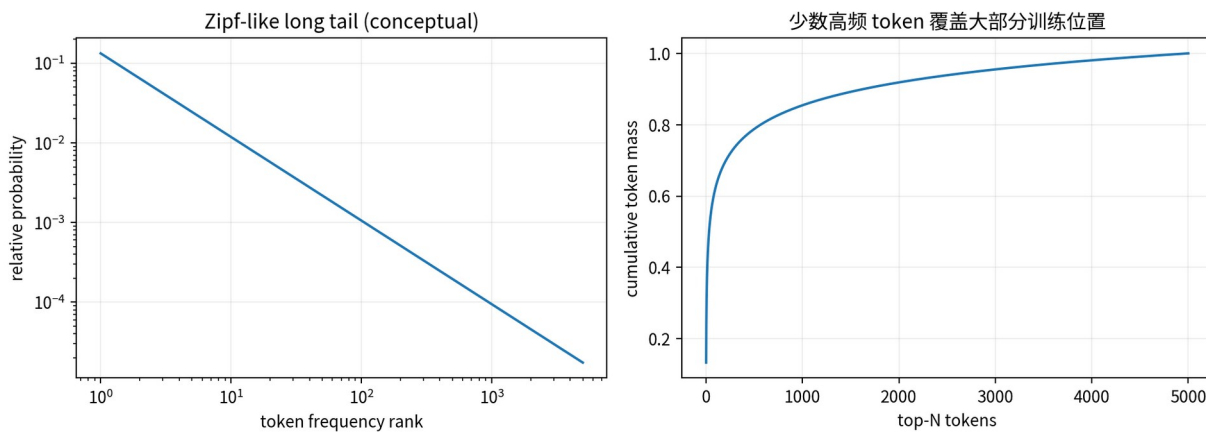


图 4 Zipf-like token frequency 的概念示意：少量 token 占据大量训练位置，尾部 token 的直接输入更新非常稀疏。

现象	机制	可能后果	监控
High-frequency row 训练充分	出现次数多	norm/方向快速稳定，也可能主导优化	exposure count, row grad norm
Rare row under-trained	lookup 次数少	新词、低资源语言、特殊符号表征不稳	frequency-bucket loss / norm
Tied 模型 rare row 仍有 output grad	softmax 对所有类别产生 $p_{j h}$ 梯度	缓解“完全没梯度”，但会产生 output-space bias	input/output grad decomposition
大词表加深长尾	新增 token 多为低频片段	token 数减少但每 row 学习样本更少	Vocab size x exposure curve

2025 年关于 vocabulary frequency imbalance 的受控研究把词表从 24K 扩到 196K，在固定 data/compute/optimization 下发现更大词表的损失收益主要来自最频繁约 2,500 个词，而稀有尾部 loss 反而升高；作者还通过约束 input/output embedding norms 验证模型在利用这种频率不平衡。[13]

2026 年 TIDE 将“token index 只在输入层注入一次”视为可重新审视的架构假设，并以 Rare Token Problem 与 Contextual Collapse 为动机，把 token identity memory 通过 router 注入多层。这代表 Embedding 设计正在从“单层查表”走向更深层 token identity 接口。[15]

6. Output Embedding / LM Head：不仅是最后一个 Linear

标准 LM Head 做 $z = W_{\text{out}} h$ ，再在 V 维上 softmax。它有三类成本： $V \times d$ 参数、每 token 的大矩阵 projection、以及 V -way softmax / cross entropy。对于大 vocab、小 hidden 的模型，LM Head 还可能成为表达能力与反向梯度的瓶颈。

问题	根源	代表研究	工程含义
Softmax bottleneck	log-probability matrix rank 受 hidden dimension 限制	Mixture of Softmaxes [5]	d 很小但目标分布高秩时，单线性 head 表达受限
Small-LM saturation	小 hidden 无法表示高秩 contextual distribution	Godey et al. 2024 [8]	$< \sim 1000$ hidden 的小模型尤其需关注
Gradient bottleneck	V -dimensional gradient 经 d -dimensional head 压缩	Godey & Artzi 2026 [17]	输出层不仅限制 logits，也可能扭曲传回 backbone 的训练信号
Large-vocab compute	projection/softmax 随 V 线性增长	Adaptive Softmax [4] 等	超大词表下需考虑 vocab parallel / approximation / factorization

“Breaking the Softmax Bottleneck” 把语言建模写成矩阵分解问题，指出标准 softmax 的表达秩受限；2024 的小模型研究进一步把 late-pretraining saturation 与该瓶颈联系起来。2026 的“LM Head is a Gradient Bottleneck” 则把问题扩展到反向传播，报告其受控实验中大量梯度范数被 output projection 压缩。[5][8][17]

边界 这些工作揭示的是标准 output head 的结构限制，并不意味着每个大模型都需要替换 softmax。对当前大型 decoder LLM， d 通常足够大；但“小模型 + 大词表 + 长训练”是值得做专门 ablation 的组合。

7. Factorized / Adaptive Embedding：什么时候值得把 v 与 d 解耦

ALBERT 的 Factorized Embedding Parameterization 把 $V \times H$ 分解为 $V \times E + E \times H$ ，使 context-independent token embedding dimension E 可以远小于 hidden H ；论文强调大 vocabulary 下这能显著减少参数。虽然 ALBERT 是 encoder 架构，但其参数经济逻辑对现代大词表 LLM 仍有参考价值。[6]

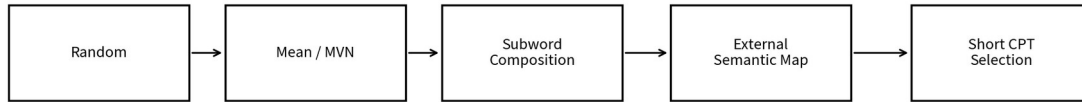
Baevski & Auli 的 Adaptive Input Representations 则根据词频给不同词分配不同表示容量，并与 adaptive softmax 结合；在当时的大词表 word-level LM 上兼顾速度和 perplexity。[4]

路线	核心思想	优点	代价
Full $V \times d$	每 token 同一 d	最简单，GPU kernel 友好	参数随 Vd 线性增长
Factorized embedding	$V \times e + e \times d, e \ll d$	解耦词表和 hidden size	增加一层 projection；是否伤大 LLM 能力需实测
Adaptive embedding	高频词高维、低频词低维	利用频率不平衡节省参数	实现复杂、对 subword 大词表未必最优
SCONE 2025	额外缓存 frequent n-gram embeddings	扩大 input representation 容量而不扩大输出 vocab	引入外部缓存与训练/服务复杂度 [12]
Per-layer embeddings	token identity 在多层重复注入	可能缓解 rare token / identity 丢失	参数与带宽巨大；Gemma 4 小模型使用 [14][15]

8. Vocabulary Expansion：新增 token 的 Embedding 怎么初始化

Continued pretraining 新语言/新领域时，经常希望扩展 tokenizer。新增 token 的问题不是 tokenizer 本身，而是 E_{in} 和 W_{out} 新 rows 没有历史训练信号。如果随机初始化，模型在开始 CPT 时会突然面对一组几何与 logit scale 不匹配类别。

Vocabulary Expansion: 从随机初始化走向结构化初始化 + 轻量 CPT 选择



新 token 初始化的目标不是“初始 loss 最低”，而是让 continued pretraining 更快进入正确几何与概率空间；2026 研究建议用极短 CPT 作为选择信号。

图 5 Vocabulary expansion 的初始化方法已经从随机向 subword composition、semantic mapping 与短 CPT 选择发展。

初始化族	做法	何时有效	注意点
Random / mean	按已有 embedding 分布采样或均值初始化	强 baseline，简单	可能初始 logits 异常；不能表示组合语义
Subword composition	用旧 tokenizer 对新 token 字符串切分，再平均/加权旧 token embedding	无需外部模型，语义与形态一致	输入与输出最优组合可能不同
External semantic mapping	跨语言词向量 / hypernetwork 映射到 LM embedding space	语言迁移	依赖外部表示与对齐质量
Convex-hull / MVN	让新 rows 落在已有 embedding 几何分布内	避免极端初始 probability	初始 loss 好不代表 CPT 最快
Short CPT selector	对多个初始化跑 50-几百步 CPT	直接测未来收敛趋势	需要额外试验预算但信号更可靠

2024 的系统比较表明，简单 multivariate initialization 与更复杂方法可以相当；OFA 通过多语言外部向量初始化 unseen subwords；2026 的系统研究比较 20+ 方法后发现 subword composition 表现很强，并指出 input 与 output embedding 的最佳初始化可以不对称，而且“初始 loss/BPB”不如少量 CPT steps 可靠。[9][11][21]

Know-how Vocabulary expansion 时，把 E_{in} 与 W_{out} 当成两个不同对象来初始化，即使原模型曾经 tied。先验证是否需要继续 tying，再决定是否共享新 rows；否则可能把一个对 input 好的初始化强加给 output classifier。

9. 多语言 / Code / 数字：Embedding Geometry 为什么由 Tokenizer 分布塑形

Embedding 不直接学习“字符的意义”，而是学习 token 在训练语料中的输入与预测统计。Tokenizer 如何把中文、阿拉伯文、代码标识符、数字、空格和特殊 token 切分，会改变每个 row 的出现次数、邻接上下文和输出目标频率，从而改变 embedding geometry。

Token 类别	典型统计问题	Embedding 层后果	建议
低资源语言 subword	出现次数少、分词碎	rows under-trained, 序列更长	监控 language-specific exposure 与 row norm
Code punctuation / whitespace	高频、结构信号强	少量符号 row 可能极重要	不要把 punctuation 当“低语义”简单降权
Numbers	数字切分策略改变组合空间	逐位 token 与整数字 token 的泛化不同	Tokenizer+Embedding+算术 eval 联合 ablation
Special tokens	频率由训练格式人工决定	system/tool/EOS 等 row 可能极高频	单独监控 norm、loss、gradient
Byte fallback tokens	只在罕见字符/乱码出现	极低频但承担 OOV 安全网	避免被 weight decay / 数据过滤完全“饿死”

因此“embedding 质量”不能只用静态 cosine nearest neighbors 判断。预训练真正关心的是：这些向量是否让后续 Transformer 更容易建立上下文表示，并让 LM Head 形成正确概率。几何可解释性只是诊断工具，不是最终目标。

10. 2026 前沿：Per-Layer Embedding 与 Token Identity Reinjection

Gemma 4 技术报告给出一个非常有代表性的变化：E2B/E4B 使用 per-layer embeddings，报告表 1 中额外 embedder 参数分别约 2.34B 与 2.82B；技术报告明确说这些额外参数来自每层 embedding，并把小模型的“effective active compute”与总参数分开。[14]

Hugging Face 的 Gemma 4 实现显示，其 per-layer embedding table 可被组织成 [vocab_size_per_layer_input, num_layers × hidden_size_per_layer_input]，在每层将 token-identity component 与 contextual projection 组合。这已经不是传统“一次 lookup 后 token id 永久丢弃”的接口。[22]

传统 single injection	Per-layer / memory reinjection
token id 只在第 0 层 lookup 一次	每层或多层重新注入 token identity
后续表示完全由 residual stream 演化	保留一个 context-free identity shortcut
参数少、系统简单	可能显著增加 embedding 参数与 memory bandwidth
rare token 只依赖最初 embedding + context	rare token 可在深层继续获得 token-specific signal
所有模型通用成熟 baseline	2026 仍属新架构方向，需要规模化 ablation

TIDE 2026 从另一路径提出 EmbeddingMemory 与深度条件 router，把 token identity 反复注入多层，并把 Rare Token Problem 作为核心动机。这和 Gemma 4 的 PLE 并非同一实现，但共同反映了“single-injection token interface”正在被重新审视。[15]

11. 分布式训练：Embedding 为什么需要 Vocab Parallel

当 $V \times d$ 达到数亿或十亿参数，一张 embedding table 可能不适合在每个 tensor-parallel rank 上完整复制。

Megatron Core 提供 VocabParallelEmbedding，沿 vocabulary dimension 切分 rows；LM Head 与 cross entropy 同样可以在 vocabulary 维分片。

系统组件	并行方式	通信关键点	Know-why
Input embedding	VocabParallelEmbedding	每个 rank 只拥有一段 vocab rows；lookup 后做 reduce / gather	参数和 optimizer state 按 V 分摊
Output logits	Column/vocab parallel projection	每 rank 输出 V/TP logits	避免先生成完整 V logits
Cross entropy	VocabParallelCrossEntropy	跨 rank 求 global max / exp sum / target logit	无需 gather 全词表 logits 再算 CE
Checkpoint	sharded state dict	保存 shard metadata 与 shape mismatch 处理	支持扩词表 / TP degree 变化
Tied weights	同一 shard 布局更容易共享	input/output parallel plan 必须一致	tying 会把系统布局也耦合起来

NVIDIA Megatron Core 当前文档将 VocabParallelEmbedding 定义为 “Embedding parallelized in the vocabulary dimension”，并提供 VocabParallelCrossEntropy 直接在分片 logits 上计算 CE。Megatron 训练参数还暴露 decoupled-lr，可对 input/output layer 使用独立学习率。[23][24][25]

系统 Know-how Embedding 是“参数很多、单次 lookup 计算很少”的特殊层；LM Head 则是“参数很多、每 token 都做大 projection”的特殊层。两者形状相似，但系统瓶颈不同，不能只用同一种并行策略思考。

12. 训练监控：不要只看 overall loss

Metric	为什么值得监控	异常信号
Embedding row norm by frequency bucket	识别高频/低频 token 的尺度漂移	rare rows norm 过低/过高；special token 极端
Input grad norm vs output grad norm	判断 tied matrix 被哪种角色支配	output/input ratio 长期过大
Per-token / bucket loss	总体 loss 会被高频 token 淹没	tail token loss 持续恶化
Token exposure count	确认 rare rows 是否真的有足够训练事件	扩词表后大量 rows 几乎没出现
LM head singular spectrum / effective rank	诊断输出空间退化与 bottleneck	小模型 late training rank collapse
Logit norm / entropy by token class	检查输出尺度与概率尖锐度	新增 token / special token probability 异常
Embedding cosine neighborhood stability	观察几何是否快速漂移	CPT 扩词表后旧 token 邻域灾难性变化

如果使用 tied embeddings，必须把“共享矩阵的总梯度”进一步分解为 input path 与 output path；否则你只能看到一个 norm，无法知道哪一侧在驱动参数。对新 token 扩展，还应单独追踪 new-vocab rows 与 original-vocab rows。

13. 严格 Ablation：怎样判断 Embedding 的更好

Ablation	必须固定	比较什么
Tied vs Untied	tokenizer、d、layers、token budget、optimizer；最好对齐总 compute	loss、downstream、rare-token bucket、memory、throughput
Vocab size x embedding	raw-text corpus 或 bytes、compute budget	token efficiency vs Vd 参数/LM-head compute
Embedding dimension factorization	backbone d、总 FLOPs	参数省多少 vs capability 是否损失
Initialization strategy	新增 token set、CPT data/order	50/100/500-step CPT curve，而非只看 step-0 loss
Per-layer reinjection	总 active compute、训练 token	small model gains 是否来自额外参数还是 identity path
Separate LR / decay	同 architecture/data	input/output convergence 与旧知识保留

实验原则 Embedding 相关实验最容易被 tokenizer token-count 变化、总参数变化和 compute 不一致混淆。尤其扩大 vocabulary 时，应至少报告 fixed tokens、fixed raw text/bytes、fixed compute 三种视角中的两种。

14. Production Blueprint：把 Embedding 当成一个可版本化子系统

层	生产设计
Tokenizer contract	固定 vocab id、special token id、normalization、byte fallback；任何变更都触发 embedding migration
Embedding schema	记录 V、d、tie policy、scale、padding_idx、dtype、factorization/per-layer settings
Init policy	from-scratch init 与 vocabulary-extension init 分开版本化
Optimization policy	LR、weight decay、gradient clipping、tied gradient decomposition；必要时 input/output decoupled LR
Parallel policy	vocab shard、TP degree、cross-entropy kernel、checkpoint layout

层	生产设计
Observability	frequency buckets、row norm、grad norm、exposure、tail loss、logit entropy、rank diagnostics
Migration	新增/删除/reorder token 禁止静默发生；必须保存 old->new id map 并重建 E_in/W_out rows
Eval gate	总体 LM loss + 多语言/code/numeric/special-token eval + rare-token slices + serving memory/throughput

一个稳健的顺序是：Tokenizer freeze -> 统计 token frequency -> 选择 V/d/tie policy -> 初始化 -> 小规模 proxy pretraining -> 监控 per-frequency embedding dynamics -> 决定是否 factorize / untie / separate LR -> 大规模训练 -> 若扩词表则走显式 migration + short-CPT selection。

一句话沉淀 Embedding 的工程对象不是“词向量本身”，而是完整 Token Interface：Tokenizer IDs -> Input Geometry -> Residual Stream -> Output Geometry -> Vocabulary Probability。任何一端变化都可能反向改变另外几端。

15. Know-Why / Know-How 总结

问题	Know-why	Know-how
为什么 Embedding 很重要？	它是离散 token 与连续计算空间的唯一入口/出口	把 E_in、LM Head、tokenizer、tie policy 一起版本化
为什么大词表会变贵？	参数和 output projection 都按 V 增长	算 Vd、2Vd、optimizer state 与 vocab-parallel 通信
为什么 tied 不一定总更好？	input 和 output 的梯度目标不同	做 tied/untied + gradient decomposition ablation
为什么 rare token 难学？	lookup 更新次数受频率直接限制	frequency buckets、exposure、tail loss、norm monitoring
为什么 LM Head 可能是瓶颈？	$d \ll V$ ，softmax 表达和梯度都经过低维线性接口	小模型/大 vocab 做 rank、saturation 与 alternative head 评估
扩词表怎么做？	新 rows 没有历史几何和 logits calibration	subword composition + norm calibration + short CPT selection
未来方向是什么？	single token-identity injection 可能不足	关注 per-layer embeddings、memory reinjection、factorized/scalable embedding

参考资料与证据等级

优先使用论文、官方技术报告、官方实现/框架文档。2026 新论文标为“前沿证据”，用于解释最新方向，不与已被广泛采用的经典设计等同。

- [1] Vaswani et al. (2017), Attention Is All You Need - 经典 / 一手论文 [Link](#)
- [2] Press & Wolf (2017), Using the Output Embedding to Improve Language Models - 经典 / 一手论文 [Link](#)
- [3] Inan et al. (2017), Tying Word Vectors and Word Classifiers - 经典 / 一手论文 [Link](#)
- [4] Baevski & Auli (2018), Adaptive Input Representations for Neural Language Modeling - 一手论文 [Link](#)
- [5] Yang et al. (2018), Breaking the Softmax Bottleneck - 经典 / 一手论文 [Link](#)
- [6] Lan et al. (2020), ALBERT: A Lite BERT - 一手论文 [Link](#)
- [7] Bertolotti & Cazzola (2024), By Tying Embeddings You Are Assuming the Distributional Hypothesis - ICML 2024 / 一手论文 [Link](#)
- [8] Godey et al. (2024), Why do small language models underperform? - COLM 2024 / 一手论文 [Link](#)
- [9] Mundra et al. (2024), Empirical Comparison of Vocabulary Expansion and Initialization - 一手论文 [Link](#)
- [10] Yamaguchi et al. (2024), Vocabulary Expansion with 0.01GB Target Text - 一手论文 [Link](#)
- [11] Liu et al. (2023), OFA: Initializing Unseen Subword Embeddings - 一手论文 [Link](#)
- [12] Yu et al. (2025), Scaling Embedding Layers in Language Models (SCONE) - 一手论文 [Link](#)
- [13] Chung & Kim (2025), Exploiting Vocabulary Frequency Imbalance in LM Pre-training - 一手论文 [Link](#)
- [14] Gemma Team (2026), Gemma 4 Technical Report - 最新官方技术报告 [Link](#)
- [15] Jaiswal et al. (2026), TIDE: Every Layer Knows the Token Beneath the Context - 前沿预印本 [Link](#)
- [16] Lopardo et al. (2026), Weight Tying Biases Token Embeddings Towards the Output Space - 前沿预印本 [Link](#)
- [17] Godey & Artzi (2026), Lost in Backpropagation: The LM Head is a Gradient Bottleneck - 前沿预印本 [Link](#)
- [18] Meta Llama 3 official model implementation - 官方实现 [Link](#)
- [19] Meta Llama 3 official model card / tokenizer vocabulary - 官方资料 [Link](#)
- [20] Hugging Face Transformers Qwen2Config - 主流实现 / 配置证据 [Link](#)
- [21] Joshi et al. (2026), Beyond Initialization Loss: Token Embedding Initialization for Vocabulary Extension - 最新预印本 [Link](#)
- [22] Hugging Face Transformers Gemma 4 modeling: per-layer embeddings - 主流实现 / 最新架构 [Link](#)
- [23] NVIDIA Megatron Core: VocabParallelEmbedding - 官方框架文档 [Link](#)
- [24] NVIDIA Megatron Core: VocabParallelCrossEntropy - 官方框架文档 [Link](#)
- [25] NVIDIA Megatron-LM training args: decoupled LR for input/output layer - 官方实现 [Link](#)

注：参数量示例按公开 *vocabulary* 与 *hidden dimension* 做矩阵尺寸计算；具体 *checkpoint* 还可能包含 *padding* 到并行友好倍数、*bias*、额外 *per-layer embedding* 或多模态 *token* 等实现差异。